

Report — VAE vs. DDPM Generative Models (GenCV003)

1. Objective

This report documents a hands-on implementation of two generative model families — a Variational Autoencoder (VAE) and a Denoising Diffusion Probabilistic Model (DDPM) — built from scratch in TensorFlow/Keras (no pre-existing generative-model libraries or pretrained backbones). Both models are trained on the same Fashion-MNIST subset and benchmarked quantitatively (FID and Inception-Score) and qualitatively, to clarify the architectural differences and practical trade-offs between the two approaches.

2. Introduction

- A VAE learns a compressed, continuous latent space and generates an image in a single decoder pass from a sampled latent vector.
- A DDPM learns to reverse a fixed, gradual noising process, generating an image through many small denoising steps starting from pure noise.

Both are trained under the same conditions (same data, same 16×16 resolution, same 100-epoch budget) so the comparison isolates architectural differences rather than dataset or protocol differences.

3. Dataset

Dataset: Fashion-MNIST — grayscale clothing images across 10 classes (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot). To keep training tractable on a single CPU, a fixed-seed (42) random subsample of 1,800 images is used, resized from 28×28 down to 16×16 via bilinear interpolation and split 85/15 into 1,530 training and 270 held-out test images. Pixel values are scaled to [0, 1] to match the decoder's sigmoid output and the binary cross-entropy reconstruction loss.

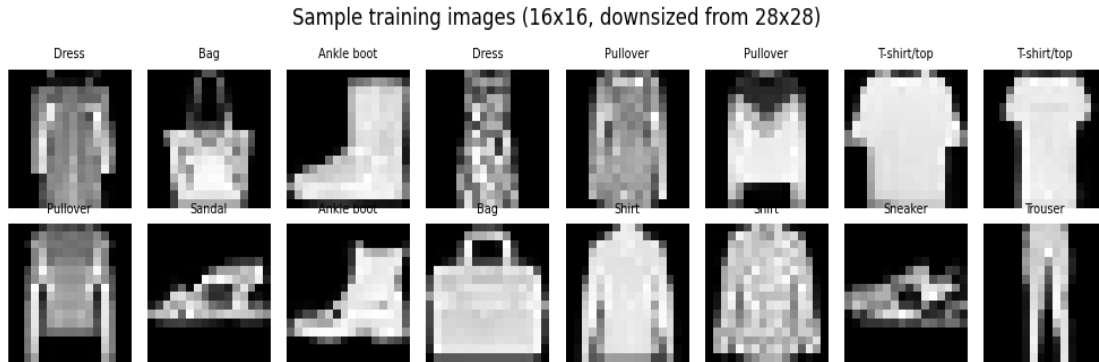


Figure 1. Random sample of the 16×16 Fashion-MNIST subset used in this run, with class labels.

4. Model 1 — Variational Autoencoder (VAE), Implemented From Scratch

4.1 Idea

The encoder maps an image to a distribution over latent vectors — a mean and log-variance (diagonal Gaussian) — and a latent vector is drawn using the reparameterization trick:

$$z = \mu + \exp(0.5 \cdot \logvar) \cdot \varepsilon, \quad \varepsilon \sim N(0, I)$$

which keeps sampling differentiable so gradients flow back through the encoder. The decoder maps z to a reconstructed image. Training minimizes the ELBO:

$$L_{\text{VAE}} = \text{BCE}(x, \hat{x}) - 0.5 \cdot \Sigma(1 + \logvar - \mu^2 - \exp(\logvar))$$

the reconstruction term (per-pixel binary cross-entropy, summed over pixels) plus a KL-divergence term pulling the latent distribution toward $N(0, I)$ — this keeps the latent space continuous and lets new images be generated by sampling z directly from $N(0, I)$.

4.2 Architecture

- Encoder: Conv2D(32, 3×3, stride 2) → Conv2D(64, 3×3, stride 2) → Flatten → two parallel Dense heads producing μ and $\log\sigma$ ($16\times 16 \rightarrow 8\times 8 \rightarrow 4\times 4$).
- Decoder: Dense($4\times 4\times 64$) → Reshape($4, 4, 64$) → Conv2DTranspose(64, stride 2) → Conv2DTranspose(32, stride 2) → Conv2D(1, sigmoid) output ($4\times 4 \rightarrow 8\times 8 \rightarrow 16\times 16$).
- Latent dimension: 16 | Optimizer: Adam, learning rate $1e-3$ | Epochs: 100, batch size 64 | Parameters: $\approx 124,700$ (computed from the architecture; not printed in the notebook).

5. Model 2 — Denoising Diffusion Probabilistic Model (DDPM), Implemented From Scratch

5.1 Idea

Forward process: Gaussian noise is added to an image gradually over T steps. Because the process is Gaussian, any step can be reached directly from the clean image x_0 in closed form:

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \text{noise}, \text{ where } \bar{\alpha}_t = \prod_{s \leq t} (1 - \beta_s)$$

Reverse process: a small ResNet-style U-Net predicts the noise added at a randomly sampled step t , given the noisy image x_t and a sinusoidal time embedding of t . Training objective is a simple regression:

$$L_{DDPM} = \text{MSE}(\text{noise_real}, \text{noise_predicted})$$

Sampling starts from pure Gaussian noise x_T and repeatedly removes a small, predicted amount of noise for $t = T, \dots, 1$ — this is the core structural difference from the VAE: generation is iterative (T sequential network calls) rather than a single forward pass.

5.2 Architecture

A compact single-level U-Net ("SimpleUNet"), built from GroupNorm + swish ResBlocks with a time-embedding bias injected mid-block:

- in_conv: Conv2D(48) at 16×16 , channels $1 \rightarrow 48$.
- down1: ResBlock(48) at $16\times 16 \rightarrow$ average-pool to 8×8 .
- down2: ResBlock(96) at 8×8 (with a 1×1 skip-projection, since channels change $48 \rightarrow 96$).
- mid: ResBlock(96) bottleneck at 8×8 .
- Upsample to $16\times 16 \rightarrow$ Conv2D(96) → concatenate with the down1 skip connection → 1×1 merge-conv back to 48 channels.
- up1: ResBlock(48) at $16\times 16 \rightarrow$ GroupNorm + SiLU → Conv2D(1) noise-prediction output.
- Time embedding: 64-dim sinusoidal embedding → 2-layer MLP to a 128-dim vector, added as a per-channel bias inside every ResBlock.
- Noise schedule: linear β from $1e-4$ to 0.02 over $T = 200$ steps | Optimizer: Adam, learning rate $1e-3$ | Epochs: 100, batch size 64 | Parameters: $\approx 540,800$ (computed from the architecture; not printed in the notebook).

6. Quantitative Benchmarking — Methodology

Standard FID/IS rely on an ImageNet-pretrained InceptionV3 network, which requires downloading weights from Google's servers — unreachable from this sandboxed, offline environment. As a substitute, a small CNN classifier is trained from scratch on this Fashion-MNIST subset (2 conv+pool blocks → Dense(64) → 10-way softmax), and:

- its 64-unit penultimate dense layer is used as the feature space for the FID calculation, in place of InceptionV3's pooling features;

- its 10-way softmax output is used as the label distribution for the Inception-Score calculation.

This preserves the mathematical form and interpretation of both metrics (lower FID = generated distribution statistically closer to real data; higher IS = sharper, more class-diverse samples), but the absolute numbers are not comparable to literature FID/IS values — only the within-run VAE-vs-DDPM comparison is meaningful. The classifier reached 99.2% training / 97.0% validation accuracy after 15 epochs.

For evaluation, both models generate as many synthetic images as there are held-out test images (270), and both metrics are computed against that real reference set. Each metric reflects a single evaluation run (no repeated sampling), so no $\pm$ uncertainty range is reported.

7. Results

7.1 Training curves

VAE: total ELBO loss dropped from 166.4 at epoch 0 to 89.6 by epoch 99 (reconstruction ≈ 82.3 , KL ≈ 7.35), converging and flattening well before the end of training.

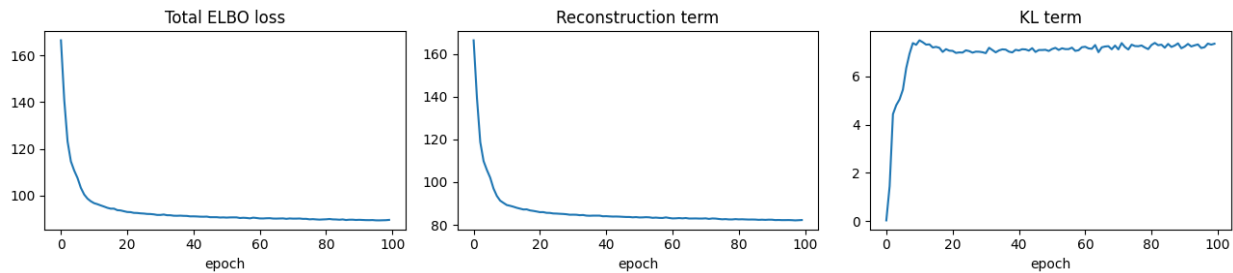


Figure 2. VAE training curves over 100 epochs — total loss, reconstruction term, and KL term.

DDPM: noise-prediction MSE dropped sharply from 0.66 at epoch 0 to about 0.11 by epoch 10, then continued a slow decline to ≈ 0.076 by epoch 99.

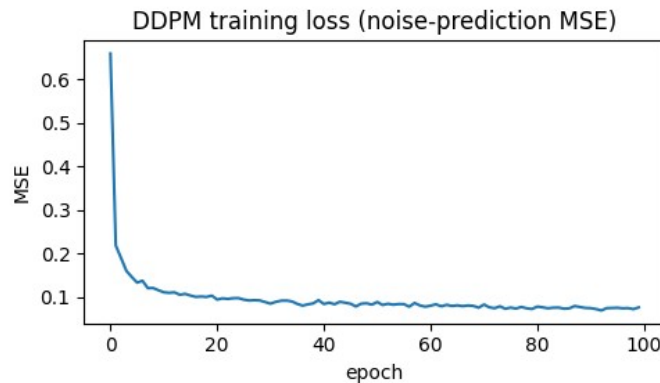


Figure 3. DDPM noise-prediction MSE loss over 100 epochs.

7.2 Quantitative results

Model	FID-style ($\downarrow$ better)	Inception-Score-style ($\uparrow$ better)
Real test set	—	6.070
VAE	34.968	3.878
DDPM	69.549	3.251

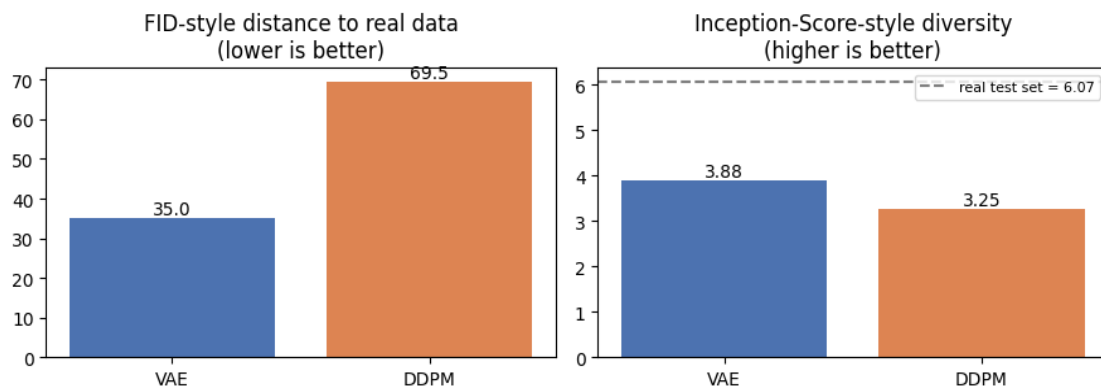


Figure 4. FID and Inception-Score, VAE vs. DDPM, on this Fashion-MNIST run.

In this run the VAE outperforms the DDPM on both metrics: a lower FID distance to real data (34.97 vs. 69.55) and a higher IS score (3.88 vs. 3.25, against a real-data ceiling of 6.07). Section 9 discusses why.

7.3 Qualitative results — model outputs

VAE: original (top) vs reconstruction (bottom)

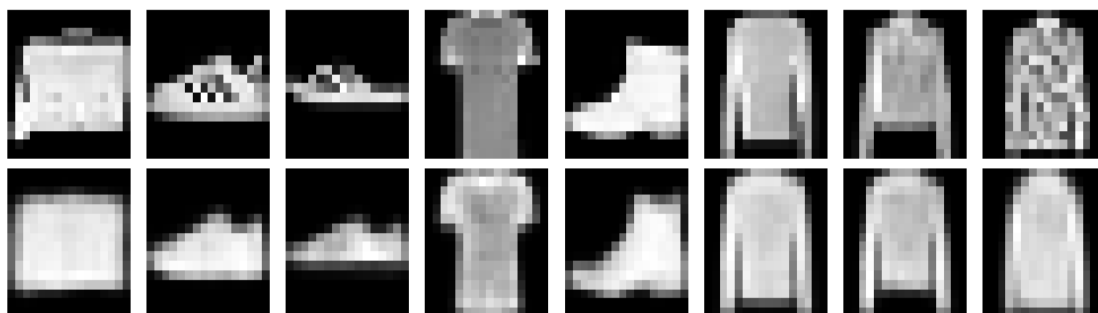


Figure 5a. VAE: original test images (top) vs. their round-trip reconstructions (bottom).

VAE: samples generated from random latent vectors

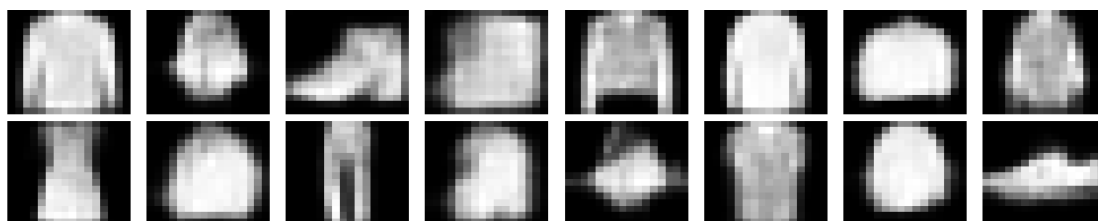


Figure 5b. VAE: brand-new samples decoded from random latent vectors $z \sim N(0, I)$.

DDPM: samples generated by reversing the diffusion process

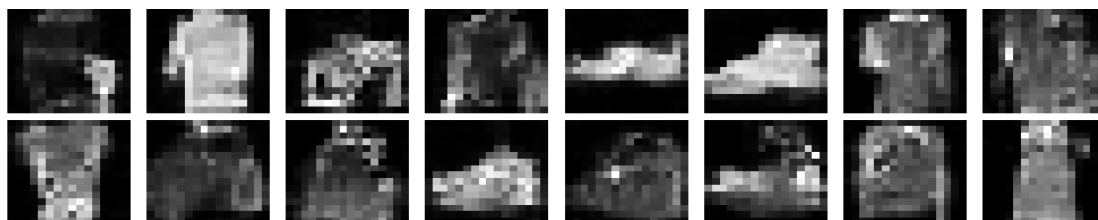


Figure 6. DDPM: novel samples generated by reversing the full $T=200$ -step diffusion process from pure Gaussian noise.

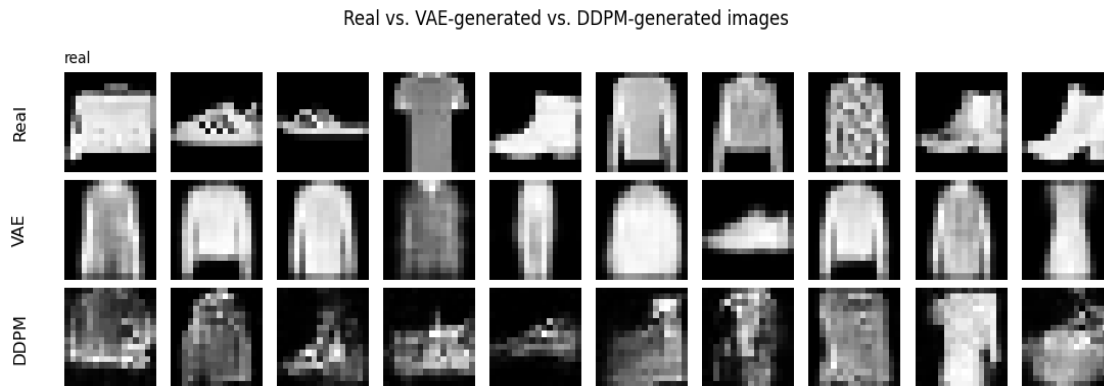


Figure 7. Real (top) vs. VAE-generated (middle) vs. DDPM-generated (bottom) — unconditional generation.

8. Qualitative Analysis — Realism, Diversity, Thematic Consistency

- **Realism:** the VAE's samples are soft and blurry (typical of a pixel-wise reconstruction loss) but consistently form a coherent garment-like silhouette. The DDPM's samples are noticeably sharper-edged than the VAE's but still show speckled noise artifacts and less coherent global shapes — the noise-predictor's loss (Figure 3) is still declining slowly at epoch 100, so the model has not fully converged.
- **Diversity:** consistent with its higher IS score (3.88 vs. 3.25), the VAE's random-latent samples (Figure 5b) vary noticeably in silhouette across the grid; the DDPM's samples (Figure 6) are more visually uniform, tending toward similar blob-like garment shapes.
- **Thematic consistency:** VAE reconstructions (Figure 5a) stay clearly "clothing-shaped" because the encoder anchors each reconstruction to a specific real input. VAE prior samples (Figure 5b) are somewhat less sharp than the reconstructions, since decoding a randomly sampled z is a harder generalization task. DDPM samples (Figure 6) are the least thematically sharp of the three, reading more as noisy textured shapes than specific garments.

9. Discussion — Why the DDPM Did Not Win Here

The literature-standard expectation is that diffusion models eventually produce sharper, more diverse samples than VAEs. That advantage is earned, not automatic — it typically requires larger datasets, more network capacity, and more diffusion/training steps than fit inside a lightweight, single-CPU run. In this setup:

- the training set is a 1,800-image subsample at 16×16 resolution, not the full 60k-image, 28×28 Fashion-MNIST split;
- the noise-predictor is a compact, single-down/up-level U-Net ($\approx 540k$ parameters) — its loss curve (Figure 3) is still trending slowly downward at epoch 100, meaning the model has not fully converged, while the VAE's loss has clearly flattened by that point;
- a diffusion model has to learn to denoise at every one of the $T=200$ noise levels using one shared network, a fundamentally harder optimization problem than the VAE's single-shot encode/decode mapping — so for a fixed, small training budget, the VAE reaches a usable state faster;
- the VAE's explicit Gaussian latent prior gives it a shortcut to reasonable-looking (if blurry) global shapes early in training, while the DDPM must learn accurate denoising across the full noise-level spectrum before its samples stop looking noisy.

So a clear VAE win on both metrics at this scale is a reasonable, explainable outcome given the compute budget — not evidence of a bug in the DDPM implementation. More training epochs, a larger U-Net, and/or the full 60k-image dataset are the standard way to close this gap, consistent with published diffusion-model results at larger scale.

10. Conclusion — Core Differences Between VAE and DDPM

Aspect	VAE	DDPM
--------	-----	------

Generation mechanism	Single forward pass through the decoder	Gradual reverse diffusion over T=200 sequential steps
Training objective	ELBO = binary-cross-entropy reconstruction + KL-to-prior	Simple regression: predict the noise added at a random step
Sample quality (this run)	Better on both metrics: FID 34.97, IS 3.88	Worse on both: FID 69.55, IS 3.25
Latent space	Explicit, continuous 16-D Gaussian	No explicit latent space — a full-resolution noisy image at each step
Best-suited use	Real-time / interactive generation	Offline generation, or real-time only with sampling accelerators

Key takeaway: the two families solve the same problem — learning to generate realistic Fashion-MNIST images — via fundamentally different mechanisms: a VAE compresses data into a structured latent distribution and decodes in one shot, while a DDPM learns to reverse a step-by-step noising process. The textbook expectation is that DDPMs eventually produce sharper, more diverse samples than VAEs, at the cost of much slower sampling — but, as this run shows concretely, that quality advantage has to be earned with sufficient data, model capacity, and training steps. At the small scale used here, the VAE is both the more compute-efficient and the higher-scoring choice; the DDPM remains the architecture with more long-run headroom once scaled up.

11. Limitations

- Metric substitution: FID/IS here use a small Fashion-MNIST-trained classifier instead of ImageNet-pretrained InceptionV3 (no internet access in this environment), so absolute values are not comparable to literature FID/IS — only the within-report VAE-vs-DDPM comparison is meaningful.
- Scale: both models are trained on a 1,800-image subsample downsized to 16×16 for CPU tractability, rather than the full 60k-image, 28×28 split.
- Single-run metrics: FID/IS are computed from one generation batch each (270 images), with no repeated sampling — so no confidence interval is reported.
- DDPM convergence: the U-Net's training loss was still declining slowly at epoch 100 (Figure 3); the DDPM results reported here are a lower bound on what the same architecture could achieve with a longer training budget.
- Unconditional generation: neither model is conditioned on class label, so neither can be asked to generate a specific garment type on demand.

12. Reproducing These Results

Both models are implemented with plain TensorFlow/Keras layers and a manual GradientTape training loop (no high-level generative-model libraries). To reproduce: install tensorflow, scipy, matplotlib, and numpy; load Fashion-MNIST via `keras.datasets.fashion_mnist`; and run the VAE cells followed by the DDPM cells top to bottom — both use a fixed seed (42) for reproducibility.